

I. SystemVerilog Fundamentals (20 Questions)

1. What are the key differences between Verilog and SystemVerilog? Why was SystemVerilog introduced?
2. Explain the logic data type in SystemVerilog and its advantages over reg and wire.
3. Differentiate between 2-state and 4-state data types in SystemVerilog. Provide examples.
4. Explain the concept of packed and unpacked arrays. When would you use one over the other?
5. What are dynamic arrays, associative arrays, and queues? Discuss their use cases and differences.
6. Explain initial and final blocks. What are their primary purposes?
7. Differentiate between task and function in SystemVerilog.
8. What are blocking and non-blocking assignments? When should each be used to avoid race conditions?
9. Explain the \$display, \$monitor, and \$strobe system tasks.
10. What is a program block and how does it help in avoiding race conditions between the testbench and DUT?
11. Explain the concept of a clocking block. What are input skew and output skew?
12. What are interfaces in SystemVerilog and why are they beneficial for verification?
13. Explain modports within an interface and their purpose.
14. What are generate blocks and when are they used in SystemVerilog?
15. Differentiate between == and === operators in SystemVerilog.
16. What is a typedef and how is it used in SystemVerilog?
17. Explain the \$root and \$unit scopes in SystemVerilog.
18. What is the \$cast system function and when is it used?
19. Describe event and fork-join constructs. Differentiate between fork-join, fork-join_any, and fork-join_none.
20. What is the timescale directive and how does it affect simulation?

II. Object-Oriented Programming (OOP) in SystemVerilog (20 Questions)

1. How does OOP benefit SystemVerilog verification?
2. Explain the concepts of class, object, and handle in SystemVerilog.
3. What is inheritance and how is it implemented in SystemVerilog?
4. Explain polymorphism and how it's achieved using virtual methods.
5. What is the difference between a virtual function and a pure virtual function?
6. Discuss the super and this keywords.
7. Explain data encapsulation and access qualifiers (public, private, protected, local).
8. What are constructors and destructors in SystemVerilog classes?
9. Differentiate between shallow copy and deep copy of objects. Why is a deep copy often necessary in testbenches?
10. What are static and automatic variables/methods in classes? What's the main difference?
11. Explain parameterized classes and their advantages.
12. What is a virtual class and when would you declare a class as virtual?
13. How do you implement callbacks in SystemVerilog? Provide a scenario where they are useful.

14. What is the factory pattern and how is it used in UVM (Universal Verification Methodology)?
15. How can you find the class name of an object handle?
16. What is randomization in SystemVerilog? Why is it crucial for verification?
17. Explain rand and randc variables. What are their differences?
18. Describe pre_randomize and post_randomize methods.
19. What are constraints in SystemVerilog? How do they control randomization?
20. Explain inline constraints and bidirectional constraints.

III. Verification Methodologies & Concepts (20 Questions)

1. What is functional coverage and code coverage? Differentiate between them.
2. Explain covergroups, coverpoints, and bins. How are they used to measure functional coverage?
3. What is cross coverage and why is it important?
4. How do you define illegal bins and ignore bins? Provide a use case.
5. What is coverage-driven verification (CDV)?
6. Explain the concept of SystemVerilog Assertions (SVA). Why are they used?
7. Differentiate between immediate assertions and concurrent assertions.
8. What are the common temporal operators used in SVA (e.g., ##, ->, |->, s_always, s_eventually)?
9. Explain the implication operator in SVA (|->, ##).
10. Can you write SVA within a class? If so, how?
11. What is the role of a scoreboard in a verification environment?
12. How do you establish communication between a monitor and a scoreboard?
13. What are mailboxes and queues? Differentiate between them and their typical use cases for inter-process communication.
14. Explain semaphores and their application in SystemVerilog for resource management.
15. What is Direct Programming Interface (DPI)? Explain DPI import and DPI export.
16. Describe the concept of virtual interfaces and why they are necessary.
17. What is constraint layering and why is it used?
18. How do you debug randomization failures in SystemVerilog?
19. Explain layered architecture in a SystemVerilog testbench.
20. What is a transaction-level model (TLM) and how does it relate to SystemVerilog verification?

IV. Advanced SystemVerilog Topics & Debugging (20 Questions)

1. What are the different types of arrays in SystemVerilog? Discuss their memory allocation and access methods.
2. How do you dynamically allocate memory for arrays?
3. Explain how to pass arrays as arguments to functions.
4. What is randsequence and its use in generating sequences of transactions?
5. How can you generate random numbers without using rand or randc keywords?
6. How do you handle asynchronous resets in RTL design and verify them in SystemVerilog?
7. Explain the concept of clock domain crossing (CDC) and common synchronization techniques.
8. How would you approach verifying a complex protocol like AXI or PCIe using SystemVerilog?

9. What are the advantages of using packages in SystemVerilog?
10. Explain forward referencing and circular dependency in SystemVerilog and how to avoid them.
11. What is the streaming operator (<<<, >>>) and its use?
12. How do you implement void functions in SystemVerilog?
13. Explain pass by value and pass by reference for arguments in functions/tasks.
14. How can you ensure that a function argument passed by reference is not modified by the function?
15. Describe the verification cycle and its closure process.
16. What is functional coverage aggregation?
17. How can you test the functionality of interrupts using functional coverage?
18. What is the difference between always_comb, always_ff, and always_latch? Why are they preferred over always @(*)?
19. How do you handle metastability in RTL design and verification?
20. Explain how to write a clock generator without using an always block.

V. Practical Scenarios & Problem Solving (20 Questions)

1. You have a multi-dimensional array, but you don't know the number of dimensions. How can you determine it?
2. How do you check if randomization was successful without using assertions?
3. Write a SystemVerilog constraint to generate an 8-bit variable with a distribution of 70% for values 0-100 and 30% for values 101-255.
4. Given a queue of elements, how would you pick elements randomly without repetition until all elements have been chosen, even if the queue contains repeated elements?
5. You have a testbench with 100% code coverage but very low functional coverage. What would be your approach to improve functional coverage?
6. Conversely, if you have 100% functional coverage but low code coverage, what would you do?
7. Design a basic SystemVerilog driver and monitor for a simple FIFO.
8. How would you write an assertion for a FIFO full condition?
9. Describe how you would debug a randomization failure.
10. Explain a scenario where you would use a semaphore in a SystemVerilog testbench.
11. How would you implement a simple scoreboard for a packet-based interface?
12. Write a SystemVerilog constraint to generate a random phone number with specific formats (e.g., operator code, local, international).
13. How do you ensure that certain address ranges are covered in your test plan (e.g., 0x2000 to 0x9000)?
14. Explain the difference between assert and assume properties in SVA. When would you use assume?
15. How would you handle a situation where you need to call a task defined in a parent object from a derived class?
16. Discuss methods to optimize an RTL design for timing closure using SystemVerilog constructs.
17. What are the challenges of designing a high-speed CDC circuit and how do you address them?
18. How do you reduce power consumption in RTL design using SystemVerilog features?
19. Explain glitch detection using SVA and provide an example.
20. You are tasked with creating a test environment for a new IP. Describe your top-level verification strategy, outlining the key SystemVerilog components and methodologies you would employ.